Szyfr Cezara i Testy Oprogramowania

Sprawozdanie z aplikacji w C# i testów xUnit

SPIS TREŚCI

1. Testy oprogramowania
1.1 Implementacja i opis programu
1.2 Opis testów jednostkowych
1.3 Opis testów integracyjnych
1.4 Wersjonowanie kodu (Git)

1. TESTY OPROGRAMOWANIA

Czym są testy jednostkowe?

Testy jednostkowe (*unit tests*) to automatyczne testy sprawdzające poprawność działania najmniejszych, pojedynczych elementów kodu źródłowego (tzw. jednostek, np. konkretnych metod lub klas) w pełnej izolacji od reszty systemu. Ich celem jest weryfikacja, czy dana część logiki biznesowej zwraca oczekiwane wyniki dla określonych danych wejściowych.

Czym są testy integracyjne?

Testy integracyjne to poziom testowania oprogramowania, w którym poszczególne moduły, klasy lub komponenty systemu są łączone i testowane jako grupa. Ich głównym celem jest wykrywanie błędów w interakcjach i komunikacji pomiędzy różnymi częściami aplikacji (np. współpraca logiki biznesowej z systemem plików, bazą danych lub zewnętrznymi bibliotekami).

Różnica między testami jednostkowymi a integracyjnymi

- **Testy jednostkowe:** Testują pojedyncze, najmniejsze fragmenty kodu (np. jedną metodę) w pełnej izolacji, bez użycia zewnętrznych zależności.
- **Testy integracyjne:** Testują współpracę wielu komponentów jednocześnie, sprawdzając, czy połączone elementy poprawnie wymieniają między sobą dane i działają zgodnie z założeniami całego systemu.

Kiedy wykonujemy testy?

Testy wykonuje się na najwcześniejszym etapie tworzenia oprogramowania – podczas implementacji kodu przez programistę (lub przed napisaniem kodu w przypadku stosowania metodyki TDD – *Test-Driven Development*). Testy uruchamiane są wielokrotnie w trakcie prac deweloperskich oraz automatycznie w procesie ciągłej integracji (CI), aby natychmiast wykryć ewentualne regresje i błędy powstałe przy modyfikacji kodu.

Przykład opisowy (bez kodu)

- **Test jednostkowy:** Sprawdzanie funkcji kalkulatora rabatowego w sklepie internetowym. Test podaje cenę początkową (100 zł) oraz wysokość zniżki (20%), po czym weryfikuje, czy obliczona kwota końcowa wynosi dokładnie 80 zł. Test nie łączy się z bazą danych ani interfejsem graficznym.
- **Test integracyjny:** Weryfikacja procesu zamówienia, w którym kalkulator rabatowy pobiera dane o zniżkach z bazy danych, przelicza kwotę, a następnie zapisuje wystawioną fakturę w formacie PDF na dysku serwera.

Opis narzędzia do testowania (xUnit)

xUnit to narzędzie typu *open-source* dedykowane dla platformy .NET, służące do pisania i automatycznego uruchamiania testów jednostkowych oraz integracyjnych. Do oznaczania metod testowych wykorzystuje atrybuty:

- `[Fact]` – oznacza klasyczny test, który uruchamia się dla jednego, stałego zestawu danych.
- `[Theory]` – umożliwia uruchomienie tej samej logiki testowej dla wielu różnych danych wejściowych przekazywanych za pomocą atrybutów `[InlineData]`.

1.1 IMPLEMENTACJA I OPIS PROGRAMU

Opis działania Szyfru Cezara

Szyfr Cezara to klasyczny algorytm szyfrowania podstawieniowego. Polega na zastąpieniu każdej litery tekstu jawnego inną literą, przesuniętą o stałą liczbę pozycji w alfabecie (tzw. klucz lub przesunięcie). Jeśli podczas przesuwania osiągnięty zostanie koniec alfabetu, algorytm zapętla się i kontynuuje odliczanie od litery "A". Program obsługuje wielkie i małe litery, a znaki niebędące literami (spacje, cyfry, znaki interpunkcyjne) pozostawia w formie niezmienionej. Deszyfrowanie polega na wykonaniu przesunięcia w przeciwnym kierunku.

Opis kodu programu

- **Klasa `CaesarCipher`:** Główna klasa aplikacji zawierająca logikę szyfrowania i deszyfrowania.
- **Metoda `Encrypt(string text, int shift)`:**
  1. *Walidacja wejścia:* Warunek sprawdza, czy przekazany ciąg znaków nie jest pustym wskaźnikiem (`null`). Jeśli tak, rzucany jest wyjątek `ArgumentNullException`.
  2. *Obliczenie przesunięcia:* Wzór sprowadza wartość przesunięcia do zakresu 0–25, co gwarantuje poprawność obliczeń dla przesunięć ujemnych lub większych od 26.
  3. *Iteracja po znakach:* Analizowany jest każdy znak tekstu z rozróżnieniem liter od pozostałych znaków.
  4. *Szyfrowanie znaków:* Przesunięcie kodu znaku w tablicy ASCII z uwzględnieniem zapętlenia alfabetu (modulo 26).
  5. *Znaki specjalne:* Znaki niebędące literami trafiają do bufora bez modyfikacji.
- **Metoda `Decrypt(string text, int shift)`:** Wywołuje metodę `Encrypt` z odwróconym kluczem przesunięcia, co odpowiada przesunięciu alfabetu w lewo i odtworzeniu tekstu oryginalnego.

1.2 OPIS TESTÓW JEDNOSTKOWYCH

Zestaw testów jednostkowych xUnit obejmuje 3 testy pozytywne oraz 1 test negatywny (zgodnie ze strukturą AAA):

1. Encrypt_StandardText_ReturnsEncryptedText (Pozytywny)
Sprawdza podstawowe działanie funkcji szyfrującej dla prostego słowa z samych wielkich liter ("HELLO", klucz 3 -> wynik "KHOOR").

2. Decrypt_EncryptedText_ReturnsOriginalText (Pozytywny)
Weryfikuje poprawność procesu deszyfrowania ("KHOOR", klucz 3 -> odtworzone "HELLO").

3. Encrypt_TextWithWrapAroundAndPunctuation_ReturnsCorrectResult (Pozytywny)
Testuje przypadki brzegowe: zawijanie alfabetu z końca na początek (np. 'Z' na 'A') oraz obecność spacji i interpunkcji ("Hello, World! Z", klucz 1 -> "Ifmmp, Xpsme! A").

4. Encrypt_NullInput_ThrowsArgumentNullException (Negatywny)
Sprawdza zachowanie aplikacji po przekazaniu wartości null. Wykorzystuje `Assert.Throws<ArgumentNullException>`.

1.3 OPIS TESTÓW INTEGRACYJNYCH

Zaimplementowano test integracyjny `Integration_EncryptText_SaveToFileAndDecryptSuccessfully` weryfikujący pełny przepływ danych i interakcję z systemem operacyjnym:

- **Zakres testu:** Integracja logiki `CaesarCipher` z systemem plików za pośrednictwem modułu `System.IO.File`.
- **Przebieg:**
  1. *Przygotowanie (Arrange):* Tworzona jest instancja klasy szyfrującej oraz plik tymczasowy na dysku za pomocą `Path.GetTempFileName()`.
  2. *Wykonanie (Act):* Tekst zostaje zaszyfrowany, zapisany do pliku tymczasowego na dysku, następnie odczytany z pliku i poddany deszyfrowaniu.
  3. *Weryfikacja (Assert):* Test sprawdza, czy tekst zaszyfrowany różni się od oryginału, czy plik zawiera poprawny szyfrogram oraz czy ostatecznie odzyskano właściwą wiadomość.
  4. *Czyszczenie (Dispose):* Dzięki `IDisposable` plik tymczasowy jest automatycznie usuwany z dysku po zakończeniu testu.

1.4 WERSJONOWANIE KODU (GIT)

Wykaz wykonanych commitów w repozytorium Git:

1. `Inicjalizacja struktury projektu w Visual Studio`
2. `Dodanie klasy CaesarCipher z walidacja danych wejsciowych`
3. `Dodanie 3 testow pozytywnych dla szyfrowania i deszyfrowania`
4. `Dodanie testu negatywnego sprawdzajacego wyjatek dla null`
5. `Dodanie testu integracyjnego wspolpracujacego z systemem plikow System.IO`